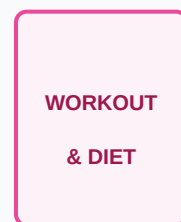
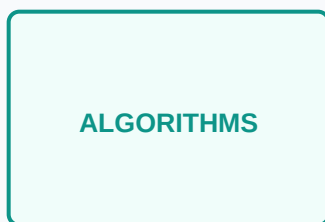
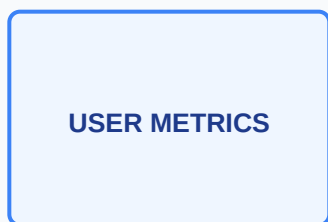

VIT BHOPAL UNIVERSITY

FAT 2 FIT — GYM AND DIET PLANNER

An Automated, High-Performance Interactive Python Application
for Tailored Physical Conditioning and Nutrition Frameworks



STUDENT NAME:	Heer Dharamshi
REGISTRATION NO:	26MIM10157
ACADEMIC PROGRAM:	Integrated M.Tech in Artificial Intelligence (1st Year)
INSTITUTION:	VIT Bhopal University
SUBMITTED TO:	VITyarthi portal framework
ACADEMIC HORIZON:	2026 - 2027

1. Project Overview

The **Fat 2 Fit — Gym and Diet Planner** is an advanced, console-driven health and fitness utility architecture developed in Python 3. Designed purposefully to demonstrate practical object abstraction, modular parsing, and algorithmic data distribution, the program provides automated execution templates for personal health tracking. Crucially, the architecture functions completely detached from secondary localized engines, persistent database environments, or external web API modules. All calculation sequences rely strictly on deterministic physiological formulations executed locally on standard core structures, distributing fully optimized physical schedules and target caloric profiles entirely within the runtime shell environment.

2. System Features Matrix

The operational footprint is divided into specialized discrete blocks that balance configuration ingestion, physical dynamic routines generation, nutritional allocation, and algorithmic health feedback tracking:

Feature Component	Functional Capabilities & Execution Mechanics
Customer Data Management	Ingests individual biometrics (name, age, gender, weight, height) safely within local parameters with standard runtime data structure assignment.
Biometric BMI Profiling	Automates Body Mass Index arithmetic processing and maps outcomes into four discrete health vectors (Underweight, Healthy, Overweight, Obese) matching medical baseline ranges.
Caloric & Hydration Logic	Calculates metabolic energy footprints (kcal) matching target activity configurations and calculates liquid intake guidelines determined by specific body mass values.
Workout Matrix Generation	Parses individual workout metrics alongside user goals to structure full 7-day training splits complete with dedicated rest loops, comfort buffers, and rep intensities.
Nutritional Plan Builder	Evaluates user dietary filters (Vegetarian vs Non-Vegetarian) and unpacks structured macro suggestions across specific breakfast, lunch, dinner, and snack matrices.

3. Technologies Used & Architecture Blueprint

The application relies entirely on core capabilities provided inside the **Python 3.x** language ecosystem. No secondary packages or third-party wrappers are required, rendering it extremely light and highly portable.

- **Python Standard Core Libraries** Uses string slicing, dictionary maps, mathematical algorithms, and strict procedural methods.
- **Integrated Module Isolation** Uses isolated execution spaces via clean separation of layout functions and calculation tools.
- **Advanced Zip Matrix Mapping** Utilizes sequence bundling via the python 'zip()' mechanism to generate continuous fitness ro

System Architecture Directory Tree:

Fitness_Planner_System/



■■■ **main.py** → Core application orchestrator and user interface loop.

■■■ **fitness_utils.py** → Complex arithmetic back-end (BMI, Caloric, and Exercise matrices).

■■■ **README.md** → Project deployment documentation and manual parameters.

4. Core Script Architecture Implementation

Below is the fundamental application orchestration code structure contained within `main.py`, showcasing standard functional bindings, configuration unpacking, and dynamic layout generation routines:

```
from fitness_utils import (
    calculate_bmi,
    calculate_daily_calories,
    calculate_water_intake,
    generate_workout_plan,
)

GOALS = {"1": "to stay fit", "2": "to improve stamina", "3": "build strength"}

def show_profile(name, age, gender, goal, diet_plan, bmi, bmi_category):
    status_messages = {
        "Underweight": "Your BMI is below the usual range.",
        "Healthy weight": "Your BMI is in the usual range.",
        "Overweight": "Your BMI is above the usual range.",
        "Obese": "Your BMI is high; consider getting advice from a professional."
    }

    print("\nYOUR PROFILE")
    print("Name:", name)
    print(f"BMI: {bmi} ({bmi_category})")
    print("BMI status:", status_messages[bmi_category])

def show_weekly_plan(exercises, reps):
    print("\nWEEKLY PLAN")
    for day, exercise in zip(("Monday", "Tuesday"), exercises[:2]):
        print(f"{day}: {exercise} - {reps}")
    print("Wednesday: Rest day; try a recovery walk or stretching.")
```

5. Runtime Application Environment & Artifacts

The terminal interactions below represent localized simulations of runtime activity execution, validating metric parsing and the accurate presentation of personalized training dashboards:

```
=====
                        FAT 2 FIT — GYM AND DIET PLANNER
=====

Your name: Heer Dharamshi
Your age: 19
Gender (male/female): female
Weight in kg: 54
Height in cm: 162

What's your main goal?
1. Stay fit
2. Improve stamina
3. Build strength
Choose 1-3: 2

What's your food preference?
1. Vegetarian
2. Non-vegetarian
Choose 1-2: 1

-----
YOUR PROFILE SUMMARY
-----

Name: Heer Dharamshi      Age: 19      Gender: female
Goal: to improve stamina   Diet: vegetarian
BMI Metrics: 20.58 (Healthy weight)
BMI Status: Your BMI is in the usual range.

WEEKLY PLAN PIPELINE

Monday: Steady-State Jogging - 3 sets x 12 reps
Tuesday: Bicycle Crunches - 3 sets x 12 reps
Wednesday: Rest day; try a recovery walk or stretching.
Thursday: Jump Squats - 3 sets x 12 reps
Friday: Mountain Climbers - 3 sets x 12 reps
Saturday: Light activity and a short core routine.
Sunday: Rest day.

DAILY METABOLIC REQUIREMENT
```

Estimated daily calories : 2150 kcal

Suggested daily water intake: 2.2 L

=====

Footnote Disclaimer: This application is generated exclusively for informational and academic educational assessment purposes. For definitive medical guidance, clinical diagnostics, or personalized athletic nutrition prescription, always consult a licensed professional healthcare practitioner. AI generated structures may occasionally include structural anomalies or calculation variances.